

# RAPIZZA Ltda.

## Documentación de Arquitectura del Sistema

---

Programación II — Cliente/Servidor

Universidad Libre Seccional Pereira · Facultad de Ingeniería · 2026-1

# 1. Arquitectura de Tres Capas

La aplicación sigue el patrón Cliente/Servidor con separación estricta de responsabilidades en tres capas independientes. Esta arquitectura es el estándar de la industria para aplicaciones web empresariales modernas.

| Capa                         | Tecnología                                            | Puerto |
|------------------------------|-------------------------------------------------------|--------|
| Presentación (Frontend)      | React 18 + Vite + TailwindCSS + Axios + Google Stitch | 5173   |
| Lógica de Negocio (Backend)  | Node.js + Express + bcrypt + JWT                      | 3001   |
| Persistencia (Base de datos) | MySQL 8 — normalización 3FN — 7 tablas                | 3306   |

## Flujo de comunicación

Cada petición sigue este ciclo:

- Usuario interactúa con la UI en React
- Axios envía HTTP Request con Authorization: Bearer <token>
- Express recibe la petición y aplica CORS + JSON parser
- verifyToken.js valida y decodifica el JWT → req.user = { id, rol }
- checkRole.js verifica que el rol tenga permiso para esa ruta
- El controlador ejecuta lógica de negocio + query SQL
- MySQL retorna los datos al controlador
- Express responde con { success, data, message }
- Axios recibe la respuesta y React actualiza el estado → re-renderiza la UI

## 2. Descripción de Componentes

### Frontend — /frontend/src/

| Archivo / Módulo                        | Responsabilidad                                                                                                               |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| pages/Login.jsx                         | Formulario de autenticación con diseño Stitch. Card roja con logo local. Spinner animado en el botón. Login con Enter o clic. |
| pages/Dashboard.jsx                     | Panel principal con stat cards en tiempo real y módulos filtrados por rol. Usa StatCard y ModuloCard.                         |
| pages/Pedidos.jsx                       | Coordinador del módulo central. Maneja estado y lógica: cargar, asignar, entregar, eliminar.                                  |
| pages/Clientes.jsx                      | CRUD de clientes con búsqueda en tiempo real. Modal para crear/editar.                                                        |
| pages/Productos.jsx                     | CRUD de pizzas y aditivos. Soft-delete para preservar historial.                                                              |
| pages/Empleados.jsx                     | Gestión de personal con conteo por tipo y filtro.                                                                             |
| pages/CuentasCobro.jsx                  | Cuentas por entregas tardías. Tabla con avatares, búsqueda y filtro.                                                          |
| components/Navbar.jsx                   | Barra de navegación reutilizable. Resalta ruta activa, muestra usuario/rol, logout con confirmación.                          |
| components/PrivateRoute.jsx             | HOC que protege rutas. Verifica sesión activa y rol antes de renderizar.                                                      |
| components/dashboard/StatCard.jsx       | Tarjeta de estadística con borde de color, icono, valor y subtítulo.                                                          |
| components/dashboard/ModuloCard.jsx     | Botón de módulo con icono en recuadro, título y descripción.                                                                  |
| components/pedidos/TablaPedidos.jsx     | Tabla de pedidos con badges de estado y botones de acción por rol.                                                            |
| components/pedidos/ModalNuevoPedido.jsx | Formulario modal de creación. Maneja su propio estado interno.                                                                |
| context/AuthContext.jsx                 | Estado global de sesión con React Context API. Provee user, token, login() y logout().                                        |
| services/api.js                         | Instancia Axios con interceptor JWT. Redirige al login si el servidor responde 401.                                           |
| services/rapizzaService.js              | Funciones por módulo: pedidoService, clienteService, productoService, empleadoService, cuentaCobroService.                    |

### Backend — /backend/

| Archivo / Módulo          | Responsabilidad                                                                                          |
|---------------------------|----------------------------------------------------------------------------------------------------------|
| server.js                 | Punto de entrada. Configura Express, CORS (origen: localhost:5173), JSON parser y monta todas las rutas. |
| config/db.js              | Pool de conexiones MySQL con mysql2/promise. Verifica la conexión al arrancar.                           |
| middleware/verifyToken.js | Extrae y valida el JWT del header Authorization. Inyecta req.user = { id, rol }.                         |

| Archivo / Módulo                 | Responsabilidad                                                                                                                                 |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| middleware/checkRole.js          | Fábrica de middlewares de rol. checkRole('admin') permite solo admins; acepta múltiples roles.                                                  |
| controllers/authController.js    | register() con bcrypt.hash (10 rounds), login() con bcrypt.compare + jwt.sign (8h), me() para verificar sesión.                                 |
| controllers/pedidoController.js  | Transacción SQL al crear pedidos. Cálculo automático de estado al entregar (>30 min → cuenta_cobro). Generación automática de cuentas de cobro. |
| controllers/rapizzaController.js | CRUD de clientes, productos (soft-delete), empleados y cuentas de cobro.                                                                        |
| routes/authRoutes.js             | POST /register, POST /login (públicas). GET /me (protegida con JWT).                                                                            |
| routes/pedidoRoutes.js           | GET, POST, PUT (asignar/entregar), DELETE y GET /estadisticas con roles.                                                                        |
| routes/rapizzaRoutes.js          | Endpoints de clientes, productos, empleados y cuentas-cobro con roles diferenciados.                                                            |

### 3. Modelo de Datos

Base de datos: rapizza\_db — normalizada en Tercera Forma Normal (3FN) — 7 tablas.

| Tabla          | PK               | Relaciones                                                         | Descripción                                                                                   |
|----------------|------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| usuarios       | id (AI)          | 1:1 → empleados                                                    | Cuentas de sistema. Rol ENUM: admin, despachador, chef, repartidor. Password con hash bcrypt. |
| empleados      | id_empledo (AI)  | 1:1 ← usuarios · 1:N → pedidos (×3)                                | Personal: 1 despachador, 3 chefs, 7 repartidores.                                             |
| clientes       | cedula (VARCHAR) | 1:N → pedidos                                                      | Datos de contacto y zona de cobertura del cliente.                                            |
| productos      | codigo (VARCHAR) | N:M → pedidos via detalle_pedido                                   | Pizzas y aditivos. Soft-delete con campo activo.                                              |
| pedidos        | numero (AI)      | N:1 clientes · N:1 empleados(×3) · 1:N detalle · 1:1 cuentas_cobro | Eje central. Estado automático al entregar.                                                   |
| detalle_pedido | id (AI)          | N:1 pedidos · N:1 productos                                        | Tabla intermedia N:M. Guarda precio al momento del pedido.                                    |
| cuentas_cobro  | id (AI)          | 1:1 ← pedidos · N:1 empleados                                      | Generada automáticamente cuando entrega > 30 minutos.                                         |

#### Lógica de negocio en la capa de datos

Al registrar hora\_entrega, el backend calcula los minutos transcurridos desde hora\_salida. Si supera 30 minutos, el estado\_pago cambia a cuenta\_cobro y se inserta automáticamente un registro en cuentas\_cobro vinculado al repartidor. Si llega a tiempo → pagado. Todo ocurre en una transacción SQL con BEGIN/COMMIT para garantizar atomicidad.

## 4. Endpoints de la API REST

### Autenticación

| Método | Endpoint           | Body                       | Auth |
|--------|--------------------|----------------------------|------|
| POST   | /api/auth/register | { usuario, password, rol } | No   |
| POST   | /api/auth/login    | { usuario, password }      | No   |
| GET    | /api/auth/me       | —                          | JWT  |

### Pedidos

| Método | Endpoint                            | Rol requerido |
|--------|-------------------------------------|---------------|
| GET    | /api/pedidos                        | JWT           |
| GET    | /api/pedidos/:id                    | JWT           |
| POST   | /api/pedidos                        | despachador   |
| PUT    | /api/pedidos/:id/asignar-repartidor | despachador   |
| PUT    | /api/pedidos/:id/entregar           | repartidor    |
| DELETE | /api/pedidos/:id                    | admin         |
| GET    | /api/pedidos/estadisticas           | despachador   |

### Dominio

| Método                    | Endpoint                     | Rol requerido     |
|---------------------------|------------------------------|-------------------|
| GET / POST                | /api/clientes                | JWT / despachador |
| PUT                       | /api/clientes/:cedula        | despachador       |
| GET / POST / PUT / DELETE | /api/productos               | JWT / admin       |
| GET / POST / PUT          | /api/empleados               | JWT / admin       |
| GET                       | /api/cuentas-cobro           | despachador       |
| PUT                       | /api/cuentas-cobro/:id/pagar | despachador       |

## 5. Seguridad y Control de Acceso

### 5.1 Autenticación con bcrypt + JWT

Las contraseñas se almacenan con bcrypt (10 rounds) — nunca en texto plano. Al login, bcrypt.compare() verifica el hash sin desencriptarlo. Si es correcto, el backend genera un JWT firmado con HS256 que expira en 8 horas. El frontend lo guarda en localStorage y lo adjunta en cada petición via Authorization: Bearer <token>. El interceptor de Axios redirige automáticamente al login si el servidor responde 401.

### 5.2 Control de acceso por rol (RBAC)

| Rol         | Permisos                                                                                   |
|-------------|--------------------------------------------------------------------------------------------|
| admin       | Acceso total: todos los módulos, eliminación de pedidos, gestión de empleados y productos. |
| despachador | Crea pedidos, asigna repartidores, gestiona clientes y cuentas de cobro. Ve estadísticas.  |
| chef        | Acceso de solo lectura al dashboard y lista de pedidos.                                    |
| repartidor  | Solo puede registrar la hora de entrega de sus pedidos asignados.                          |

### 5.3 Variables de entorno

Todas las credenciales se configuran en .env (excluido del repositorio con .gitignore). El repositorio incluye .env.example con los nombres de las variables sin valores reales. El JWT\_SECRET nunca aparece en el código fuente.

## 6. Justificación de Decisiones Técnicas

| Decisión                                | Justificación                                                                                                                                               |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Google Stitch para el diseño UI         | Herramienta recomendada por el docente. Generó el diseño base adaptado a JSX de React. El logo se almacena localmente en assets/ para independencia de red. |
| Componente Navbar reutilizable          | Centraliza el logout, resalta la ruta activa con useLocation() y filtra los links por rol. Evita repetir código en cada página.                             |
| Subcomponentes para Dashboard y Pedidos | Separación de responsabilidades: StatCard, ModuloCard, TablaPedidos y ModalNuevoPedido hacen una sola cosa cada uno.                                        |
| mysql2 directo sin Sequelize            | Mayor control sobre las queries SQL, facilita transacciones explícitas (BEGIN/COMMIT) y reduce la curva de aprendizaje.                                     |
| Transacción SQL al crear pedidos        | Crear un pedido involucra N inserciones. Si falla alguna, el rollback garantiza que no queden datos inconsistentes.                                         |
| Lógica de 30 minutos en el backend      | La regla de negocio se implementa en el servidor, no en el frontend, para que no pueda ser manipulada desde el navegador.                                   |
| Soft-delete en productos                | Eliminar físicamente rompería referencias históricas en detalle_pedido. El campo activo=FALSE preserva la integridad del historial.                         |
| SweetAlert2 para confirmaciones         | Requerido por RF-04. Reemplaza alert() y window.confirm() nativos con confirmaciones personalizadas y más seguras.                                          |
| Proxy de Vite hacia el backend          | vite.config.js redirige /api a localhost:3001, evitando errores de CORS y permitiendo URLs relativas en Axios.                                              |